

24. Plusieurs cas : else if, puis switch

« else if » range des intervalles ; « switch » aiguille sur des valeurs exactes.

1. Ce qu'il faut comprendre

Une chaîne `if ... else if ... else` teste **dans l'ordre**, et s'arrête au **premier** cas vrai. C'est ce qui permet d'écrire `score < 120` sans répéter `score >= 60` : si l'on arrive là, c'est que le premier test a déjà échoué.

L'ordre compte donc. Mets `score < 120` en premier, et « BRONZE » ne s'affichera plus jamais : un score de 10 est aussi plus petit que 120.

`switch (valeur)` compare **une** valeur à des cas **exacts** : `case 0:`, `case 1:`. On saute directement au bon cas, puis `break;` sort du `switch`. **Oublier** `break` fait tomber dans le cas suivant — c'est permis en C++, et c'est presque toujours un bogue. `default:` attrape tout ce qui n'a pas de case.

Quand choisir lequel ? `else if` pour des **intervalles** (moins de 60, moins de 120...), `switch` pour une liste de **valeurs précises** — un numéro de monde, un écran de jeu, une direction.

Toutes les réponses font **six lettres**, espaces compris : « OR ». Chacune recouvre exactement la précédente.

À toi : ajoute une médaille « PLATINE » au-dessus de 200.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <diviser>` → [0.8.5. #include <diviser> — a / b](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1016 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	199 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
Diviser	17 o	← main() — divise — le processeur ne sait pas le faire
CopierTuiles	85 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	480 o	← CopierTuiles — 30 lettres sur 43
"SCORE:"	6 o	← texte() — un texte gravé
"MEDAILLE:"	9 o	← texte() — un texte gravé
"MONDE:"	6 o	← texte() — un texte gravé
"BRONZE"	6 o	← texte() — un texte gravé

```

"ARGENT"      6 o ← texte() – un texte gravé
"OR   "      6 o ← texte() – un texte gravé
"PLAINE"     6 o ← texte() – un texte gravé
"GROTTE"     6 o ← texte() – un texte gravé
"CHATEAU"    7 o ← texte() – un texte gravé
"?   "      7 o ← texte() – un texte gravé

```

Lettres gravées : ABCDEGHILMNOPRSTUZ0123456789?:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```

texte() × 10   110 o sur place, dans main() + EcrireTexte (10 o)
nombre() × 1   10 o sur place, dans main() + EcrireNombre (33 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : un score monte tout seul. Selon sa valeur, on affiche
// une médaille (BRONZE, ARGENT, OR) et un monde (PLAINE, GROTTE, CHATEAU).
//
// Ce qui est nouveau :
// else if = « sinon, si ... » : on essaie les cas l'un après l'autre.
// switch = on saute directement au « case » qui a la bonne valeur.

#include <texte>      // écrit un texte à l'écran
#include <nombre>     // écrit un nombre en chiffres
#include <diviser>    // a / b, sauf par 1, 2, 4, 8, 16... écrits en clair

```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {                                // Le jeu commence ici.
    uint8_t score = 0;                      // Le score part de 0.

    texte(1, 2, "SCORE:");                  // Les trois titres, écrits une fois.
    texte(1, 5, "MEDAILLE:");
    texte(1, 8, "MONDE:");

    while (true) {                          // Répète pour toujours :
        image();                            // attend l'image suivante (60 par seconde).
        score++;                            // Le score monte tout seul : + 1 à chaque image.
        nombre(12, 2, score);               // On l'affiche.

        // Des intervalles : else if, dans l'ordre.
        // On lit de haut en bas ; dès qu'un test est vrai, on fait son bloc et on s'arrête.
        if (score < 60) {                    // De 0 à 59 :
            texte(12, 5, "BRONZE");
        } else if (score < 120) {             // Sinon, de 60 à 119 (on sait déjà qu'il est >= 60) :
            texte(12, 5, "ARGENT");
        } else {                             // Sinon, tout le reste (120 et plus) :
            texte(12, 5, "OR   ");           // Les espaces effacent la fin de ARGENT.
        }

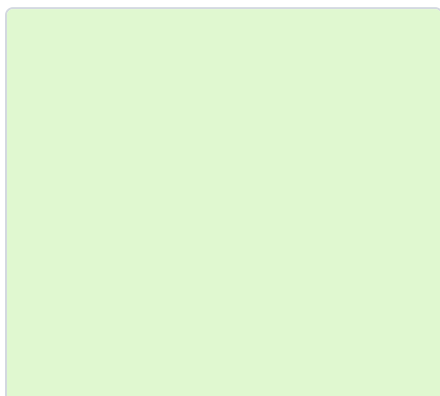
        // Des valeurs exactes : switch.
        // score / 60 vaut 0 (score 0 à 59), 1 (60 à 119), 2 (120 à 179), 3 (180 à 239) ou 4.
        switch (score / 60) {
            case 0: texte(12, 8, "PLAINE"); break; // si ça vaut 0. « break » = sortir du switch.
            case 1: texte(12, 8, "GROTTE"); break; // si ça vaut 1.
            case 2: texte(12, 8, "CHATEAU"); break; // si ça vaut 2.
            default: texte(12, 8, "?   "); break; // default = « aucun des cas au-dessus ».
        }

        // Après 255, le score repart à 0 (un uint8_t ne va que jusqu'à 255) :
        // on retombe sur BRONZE et PLAINE.
    }
}

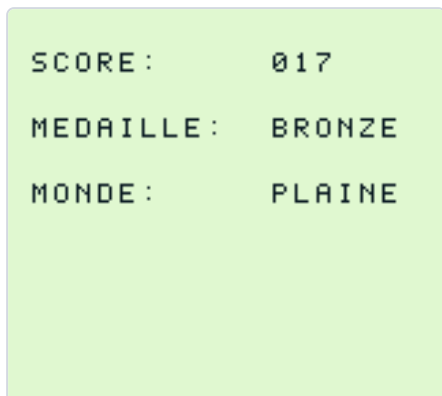
```

3. Ce qui se passe, étape par étape

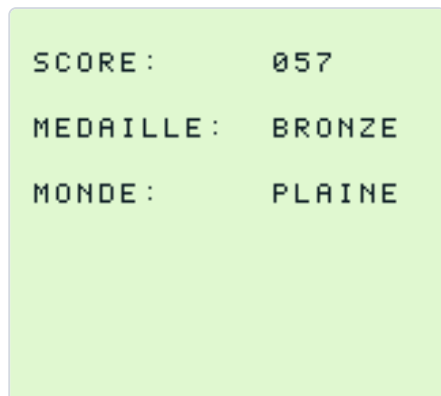
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



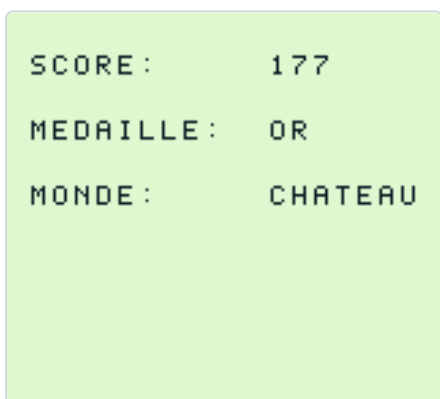
la toute première image — la console vient de démarrer — image 1



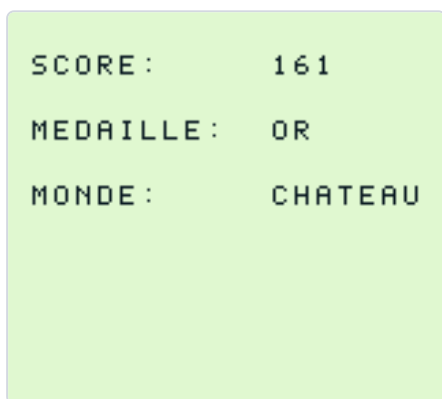
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le score monte : BRONZE/PLAINE, puis ARGENT/GROTTE, puis OR/CHATEAU.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au début : BRONZE, PLAINE (BRONZE, PLAINE)
- ✓ une seconde plus tard : ARGENT, GROTTE (ARGENT, GROTTE)
- ✓ encore une : OR, CHATEAU (OR , CHATEAU)

Ensuite : **24.1. Chapitre 5 — Les boucles : répéter : ce qui vient** — chapitre 5 : Les boucles : répéter. · gameboy3 — écrire une cartouche Game Boy en C++